Package 'Phenelope'

October 13, 2026

Type Package

Title LLM-based concept set builder

Version 0.2.0

Author Joel N. Swerdel

Maintainer Joel N. Swerdel <jswerdel@its.jnj.com>

Description A package that uses an LLM for building a concept set for use in cohort definitions.

License Apache License 2.0

VignetteBuilder knitr

Depends R (>= 4.2.0)

Imports dplyr (>= 1.1.4),
DatabaseConnector (>= 6.3.0),
httr,
rJava,
Capr (>= 3.0.0),
jsonlite,
ellmer,
SqlRender,
rlang,
CirceR,
checkmate,
R6,
readr

Suggests knitr,
rmarkdown,
testthat,
withr,
keyring,
Eunomia

Remotes ohdsi/Capr

URL https://ohdsi.github.io/Phenelope, https://github.com/OHDSI/Phenelope

BugReports https://github.com/OHDSI/Phenelope/issues

Encoding UTF-8

Config/roxygen2/version 8.1.0

Roxygen list(markdown = TRUE)

Contents

asConcepts . . . 2
ConceptAdjudicator . . . 3
ConceptRecommender . . . 4
condenseConceptSet . . . 5
createConceptSet . . . 5
createDrugConceptSet . . . 7
DefaultConceptAdjudicator . . . 8
DefaultSeedConceptFinder . . . 10
fetchCondenserConceptSetData . . . 11
getDomain . . . 12
getDomainSettings . . . 13
HecateConceptRecomender . . . 13
SeedConceptFinder . . . 14
validateConcepts . . . 15

Index **17**

`asConcepts` *Convert data frame to Concepts*

Description

The `Concepts` class enforces several properties of a data frame. First of all, it *must* have columns:

- conceptId
- conceptName
- vocabularyId
- domainId
- conceptClassId
- origin
- status

Second, the `origin` column *must* be one of these values:

- SEED
- DESCENDANT
- RECOMMENDED

Third, the `status` column *must* be one of these values:

- UNADJUDICATED
- APPROVED
- REJECTED
- BELOW_MIN_COUNT

Finally, the `conceptId` column cannot have duplicates.

Usage

```
asConcepts(dataFrame, origin = NULL, status = NULL)
```

Arguments

dataFrame	Input data frame
origin	Optional: the origin to use for all concepts in the input. Can be 'SEED', 'DESCENDANT', or 'RECOMMENDED'.
status	Optional: the status to use for all concepts in the input. Can be 'UNADJUDICATED', 'APPROVED', 'REJECTED', or 'BELOW_MIN_COUNT.

Value

An object of type Concepts.

See Also

validateConcepts()

ConceptAdjudicator *An abstract class for a concept adjudicator.*

Description

Adjudicate concepts

Methods

Public methods:

- ConceptAdjudicator$adjudicateConcepts()
- ConceptAdjudicator$clone()

ConceptAdjudicator$adjudicateConcepts():

Usage:

```
ConceptAdjudicator$adjudicateConcepts(
  concepts,
  name,
  clinicalDefinition = NULL,
  llmClient,
  costTracker = NULL
)
```

Arguments:

concepts An object of type Concepts as created using asConcepts().

name The name of the concept set.

clinicalDefinition Optional: a clinical definition of the concept set target.

llmClient An ellmer chat object to connect to a large language model, as created for example using ellmer::chat_ollama().

costTracker An environment with an amount variable for keeping track of the total LLM costs.

Returns: The same concepts as provided in the input, but with their `status` tag set to either 'APPROVED' or 'REJECTED', and an additional `rationale` column capturing the LLM rationale.

`ConceptAdjudicator$clone()`: The objects of this class are cloneable with this method.

Usage:

```
ConceptAdjudicator$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

`ConceptRecommender` *The abstract concept recommender class Recommend concepts*

Description

The abstract concept recommender class Recommend concepts

Methods

Public methods:

- `ConceptRecommender$recommendConcepts()`
- `ConceptRecommender$clone()`

`ConceptRecommender$recommendConcepts()`:

Usage:

```
ConceptRecommender$recommendConcepts(
  conceptIds,
  domainSettings,
  excludedVocabularyIds = NULL,
  connection,
  vocabDatabaseSchema,
  iteration
)
```

Arguments:

`conceptIds` A set of concept IDs to find related concepts for.

`domainSettings` An object of type `DomainSettings` as created by `getDomainSettings()`.

`excludedVocabularyIds` Character vector of OHDSI Vocabulary names to exclude.

`connection` An open connection to a database server, as created by `DatabaseConnector::connect()`.

`vocabDatabaseSchema` The database and or schema where the OHDSI Vocabulary tables are loaded.

`iteration` the current pass through the recommender

Returns: An object of type `Concepts` containing the recommended concepts. Overlap with the input concepts is *not* removed.

`ConceptRecommender$clone()`: The objects of this class are cloneable with this method.

Usage:

```
ConceptRecommender$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

condenseConceptSet *Condense concept set*

Description

Condense concept set

Usage

```
condenseConceptSet(conceptSetData)
```

Arguments

`conceptSetData` An object of type `ConceptData` as created by `fetchConceptSetData`.

Value

A tibble with a representation of the optimal concept set expression

createConceptSet *Create a concept set*

Description

Follows the following flow:

1. Determine the domain of the input, and retrieves corresponding concept filters and settings.
2. If no seed concept IDs are provided, run the seed concept finder to find seed concepts.
3. Run the concept recommender to get recommended concepts.
4. Use the concept adjucator to adjudicate whether concepts belong to the concept set or not.
5. Convert the approved concepts to a concept set expression. If `condenseConceptSet = TRUE`, a parsimoneous representation will be generated.

Steps 3 and 4 are executed twice.

Throws an error of class `NoApprovedConceptsError` if none of the recommended concepts are approved during adjudiction.

Usage

```
createConceptSet(
  name,
  seedConceptIds = NULL,
  clinicalDefinition = NULL,
  llmClient,
  connectionDetails,
  vocabDatabaseSchema,
  tempEmulationSchema = getOption("sqlRenderTempEmulationSchema"),
  cacheFolder = NULL,
  excludedVocabularyIds = c("ICDO3"),
```

```
  seedConceptFinder = DefaultSeedConceptFinder$new(),
  conceptRecommender = HecateConceptRecomender$new(),
  conceptAdjudicator = DefaultConceptAdjudicator$new(),
  condenseConceptSet = TRUE
)
```

Arguments

`name`	The name of the concept set to create. This should be an informative name reflecting the concept set target (the idea expressed by the concept set).
`seedConceptIds`	Optional: a set of one or more concept IDs that resemble the concept set target.
`clinicalDefinition`	Optional: a clinical definition of the concept set target.
`llmClient`	An `ellmer` chat object to connect to a large language model, as created for example using `ellmer::chat_ollama()`.
`connectionDetails`	An object with details for connecting to a database server, as created using `DatabaseConnector::createConnectionDetails()`.
`vocabDatabaseSchema`	The database and or schema where the OHDSI Vocabulary tables are loaded.
`tempEmulationSchema`	For database platforms that do not natively support temp tables, a database schema where the user has write access.
`cacheFolder`	Optional: Folder where intermediary results can be stored.
`excludedVocabularyIds`	Character vector of OHDSI Vocabulary names to exclude.
`seedConceptFinder`	An object of class `SeedConceptFinder` for finding seed concepts when not provided.
`conceptRecommender`	An object of class `ConceptRecommender` for recommending additional concepts.
`conceptAdjudicator`	An object of class `ConceptAdjudicator` for adjudicating recommended concepts.
`condenseConceptSet`	Condense the resulting concept set?

Value

A concept set expression

`createDrugConceptSet` *Create a drug concept set*

Description

Create a drug concept set

Usage

```
createDrugConceptSet(
  name,
  clinicalDefinition = NULL,
  llmClient,
  connectionDetails,
  vocabDatabaseSchema,
  tempEmulationSchema = getOption("sqlRenderTempEmulationSchema"),
  cacheFolder = NULL,
  fuzzyVocabSearchType = "HECATE"
)
```

Arguments

`name`	The name of the concept set to create. This should be an informative name reflecting the concept set target (the idea expressed by the concept set).
`clinicalDefinition`	Optional: a clinical definition of the concept set target.
`llmClient`	An `ellmer` chat object to connect to a large language model, as created for example using `ellmer::chat_ollama()`.
`connectionDetails`	An object with details for connecting to a database server, as created using `DatabaseConnector::createConnectionDetails()`.
`vocabDatabaseSchema`	The database and or schema where the OHDSI Vocabulary tables are loaded.
`tempEmulationSchema`	For database platforms that do not natively support temp tables, a database schema where the user has write access.
`cacheFolder`	Optional: Folder where intermediary results can be stored.
`fuzzyVocabSearchType`	Type of fuzzy vocabulary search. Currently only 'HECATE' is supported.

Value

A concept set expression

DefaultConceptAdjudicator *The default concept adjudicator using the LLM and prompts.*

Description

DefaultConceptAdjudicator constructor

This adjudicator implements a 2-stage concept adjudication process using prompts. If the number of input concepts is greater than or equal to `nForQuickScreen`, the quickscreen stage is entered. In this stage, `quickScreenBatchSize` concepts at a time are fed to the LLM using the `quickScreenPrompt` and `quickScreenSystemPrompt` prompts. Any remaining concepts go on to stage 2, where `batchSize` concepts are fed to the LLM using the `prompt` and `systemPrompt` prompts.

The following placeholders in the prompts will be replaced with their actual values when adjudicating:

- **%name%**: The name of the concept set (the concept set target).
- **%definition%**: The clinical definition of the concept set. If not provided during adjudication, the entire line with this placeholder will be deleted.
- **%concepts%**: A JSON string representing the candidate concepts.

The following JSON output formats is expected for the main concept adjudication. This should include **all concepts** from the input:

```
[
  {
    "conceptId": <Candidate Concept id>,
    "conceptName": "<Candidate Concept name>",
    "decision": "<KEEP or REMOVE>",
    "rationale": "<Brief rational for the decision>"
  }
]
```

The following JSON output formats is expected for the quick screening. This should include *only concepts to remove**:

```
[
  {
    "conceptId": <Candidate Concept id>,
    "conceptName": "<Candidate Concept name>",
   "rationale": "Brief explanation of why the concept does not imply the Target Term"
  }
]
```

If a prompt is not provided in this constructor it will assume the default value included in this package. To disable the quick screen stage, set `nForQuickScreen = 999999`.

Super class

`ConceptAdjudicator` -> `DefaultConceptAdjudicator`

Methods

Public methods:

- DefaultConceptAdjudicator$new()
- DefaultConceptAdjudicator$adjudicateConcepts()
- DefaultConceptAdjudicator$clone()

`DefaultConceptAdjudicator$new()`:

Usage:

```
DefaultConceptAdjudicator$new(
  batchSize = 20,
  nForQuickScreen = 500,
  quickScreenBatchSize = 200,
  maxN = 5000,
  prompt = NULL,
  systemPrompt = NULL,
  quickScreenPrompt = NULL,
  quickScreenSystemPrompt = NULL
)
```

Arguments:

`batchSize` The number of concepts to adjudicate at once.

`nForQuickScreen` The minimum number of concepts to trigger the quick screening.

`quickScreenBatchSize` The number of concepts to enter quick screening at once.

`maxN` If the number of concepts to adjudicate is greater than this number, an error of class `TooManyConceptsError` is thrown.

`prompt` The prompt for the main concept adjudication. See Details for requirements

`systemPrompt` The system prompt for the main concept adjudication. See Details for requirements

`quickScreenPrompt` The prompt for the quick screening. See Details for requirements

`quickScreenSystemPrompt` The system prompt for the quick screening. See Details for requirements

Returns: This is the constructor.

`DefaultConceptAdjudicator$adjudicateConcepts()`: Adjudicates concepts using the default LLM implementation.

Usage:

```
DefaultConceptAdjudicator$adjudicateConcepts(
  concepts,
  name,
  clinicalDefinition = NULL,
  llmClient,
  costTracker = NULL
)
```

Arguments:

`concepts` An object of type `Concepts` as created using `asConcepts()`.

`name` The name of the concept set.

`clinicalDefinition` Optional: a clinical definition of the concept set target.

`llmClient` An `ellmer` chat object to connect to a large language model, as created for example using `ellmer::chat_ollama()`.

`costTracker` An environment with an `amount` variable for keeping track of the total LLM costs.

`DefaultConceptAdjudicator$clone()`: The objects of this class are cloneable with this method.

Usage:

```
DefaultConceptAdjudicator$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

`DefaultSeedConceptFinder`

The default seed concept finder

Description

DefaultSeedConceptFinder constructor

Super class

`SeedConceptFinder` -> `DefaultSeedConceptFinder`

Methods

Public methods:

- `DefaultSeedConceptFinder$new()`
- `DefaultSeedConceptFinder$findSeedConcepts()`
- `DefaultSeedConceptFinder$clone()`

`DefaultSeedConceptFinder$new()`:

Usage:

```
DefaultSeedConceptFinder$new(
  addSynonyms = TRUE,
  maxN = 25,
  minCount = 0,
  fuzzyVocabSearchType = "HECATE",
  adjudicateFuzzySearchResults = TRUE
)
```

Arguments:

`addSynonyms` Generate synonyms of the concept target name before search for matching concepts?

`maxN` Maximum number of seed concepts to return.

`minCount` Minimum number of record counts for a concept to be included.

`fuzzyVocabSearchType` Type of fuzzy vocabulary search. Currently only 'HECATE' is supported.

`adjudicateFuzzySearchResults` Perform a first pass adjudication on the fuzzy vocabulary search results to remove concepts that are obviously wrong?

Returns: An object of type `DefaultSeedConceptFinder`

`DefaultSeedConceptFinder$findSeedConcepts()`: Default implementation of the concept finder.

Usage:

```
DefaultSeedConceptFinder$findSeedConcepts(
  name,
  clinicalDefinition = NULL,
  llmClient = NULL,
  costTracker = NULL,
  domainSettings = NULL,
  excludedVocabularyIds = NULL,
  connection = NULL,
  vocabDatabaseSchema = NULL
)
```

Arguments:

`name` The name to use to find concepts.

`clinicalDefinition` Optional: a clinical definition of the concept set target.

`llmClient` An `ellmer` chat object to connect to a large language model, as created for example using `ellmer::chat_ollama()`.

`costTracker` An environment with an `amount` variable for keeping track of the total LLM costs.

`domainSettings` An object of type `DomainSettings` as created by `getDomainSettings()`.

`excludedVocabularyIds` Character vector of OHDSI Vocabulary names to exclude.

`connection` An open connection to a database server, as created by `DatabaseConnector::connect()`.

`vocabDatabaseSchema` The database and or schema where the OHDSI Vocabulary tables are loaded.

`DefaultSeedConceptFinder$clone()`: The objects of this class are cloneable with this method.

Usage:

```
DefaultSeedConceptFinder$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

`fetchCondenserConceptSetData` *Fetch data for concept set condensation*

Description

Fetch data for concept set condensation

Usage

```
fetchCondenserConceptSetData(
  conceptSetExpression,
  connectionDetails = NULL,
  connection = NULL,
  cdmDatabaseSchema,
  tempEmulationSchema = getOption("sqlRenderTempEmulationSchema"),
  excludedVocabularies = c("ICDO3")
)
```

Arguments

conceptSetExpression	The concept set expression to condense. This can be either a JSON expression or the output of ROhdsiWebApi::getConceptSetDefinition.
connectionDetails	The output of DatabaseConnector::createConnectionDetails(). Is ignored if connection is provided.
connection	A DatabaseConnector connection. Can be NULL if connectionDetails is provided.
cdmDatabaseSchema	A database schema holding the OHDSI Vocabulary tables.
tempEmulationSchema	For database platforms that do not natively support temp tables, a database schema where the user has write access.
excludedVocabularies	Vocabularies not to be included in the condensing function

Value

A list containing the three inputs for condenseConceptSet().

---

getDomain *Get the domain of a concept set target*

Description

Get the domain of a concept set target

Usage

```
getDomain(name, llmClient, costTracker = NULL)
```

Arguments

name	The name of the concept set.
llmClient	An ellmer chat object to connect to a large language model, as created for example using ellmer::chat_ollama().
costTracker	An environment with an amount variable for keeping track of the total LLM costs.

Value

An uppercase character string with the name of the domain (e.g. 'CONDITION').

`getDomainSettings` *Get the settings for a specific domain.*

Description

Get the settings for a specific domain.

Usage

```
getDomainSettings(domain)
```

Arguments

`domain`	The name of a domain (all caps), e.g. 'CONDITION'.

Value

An object of type `DomainSettings`.

See Also

`getDomain()`

`HecateConceptRecomender` *Concept recommender implementation using the Hecate Phoebe search.*

Description

HecateConceptRecomender constructor

Super class

`ConceptRecommender` -> `HecateConceptRecomender`

Methods

Public methods:

- `HecateConceptRecomender$new()`
- `HecateConceptRecomender$recommendConcepts()`
- `HecateConceptRecomender$clone()`

`HecateConceptRecomender$new()`:

Usage:

```
HecateConceptRecomender$new(minCount = 0, keepLowCountConcepts = TRUE)
```

Arguments:

`minCount` Minimum record count for concepts to be considered for adjudication.

keepLowCountConcepts If TRUE concepts are kepts with status BELOW_MIN_COUNT. These concepts will be included in the final concept set only if they are descendants of concepts that are explicitly included in the concept set.

Returns: An object of type HecateConceptRecomender.

HecateConceptRecomender$recommendConcepts(): Recommends concepts using the Hecate Phoebe implementation.

Usage:

```
HecateConceptRecomender$recommendConcepts(
  conceptIds,
  domainSettings,
  excludedVocabularyIds = NULL,
  connection,
  vocabDatabaseSchema,
  iteration
)
```

Arguments:

conceptIds A set of concept IDs to find related concepts for.

domainSettings An object of type DomainSettings as created by getDomainSettings().

excludedVocabularyIds Character vector of OHDSI Vocabulary names to exclude.

connection An open connection to a database server, as created by DatabaseConnector::connect().

vocabDatabaseSchema The database and or schema where the OHDSI Vocabulary tables are loaded.

iteration the current pass through the recommender

HecateConceptRecomender$clone(): The objects of this class are cloneable with this method.

Usage:

```
HecateConceptRecomender$clone(deep = FALSE)
```

Arguments:

deep Whether to make a deep clone.

SeedConceptFinder *An abstract class for a seed concept finder.*

Description

Find seed concepts

Methods

Public methods:

- SeedConceptFinder$findSeedConcepts()
- SeedConceptFinder$clone()

SeedConceptFinder$findSeedConcepts(): Return seed concepts given a concept set name and optionally a clinical definition. The llmClient is only used when addSynonyms and/or adjudicateFuzzySearchResults is TRUE in the arguments.

Throws an error of class NoSeedConceptsError when no seed concepts are found.

Usage:

```
SeedConceptFinder$findSeedConcepts(
  name,
  clinicalDefinition = NULL,
  llmClient = NULL,
  costTracker = NULL,
  domainSettings = NULL,
  excludedVocabularyIds = NULL,
  connection = NULL,
  vocabDatabaseSchema = NULL
)
```

Arguments:

`name` The name to use to find concepts.

`clinicalDefinition` Optional: a clinical definition of the concept set target.

`llmClient` An `ellmer` chat object to connect to a large language model, as created for example using `ellmer::chat_ollama()`.

`costTracker` An environment with an `amount` variable for keeping track of the total LLM costs.

`domainSettings` An object of type `DomainSettings` as created by `getDomainSettings()`.

`excludedVocabularyIds` Character vector of OHDSI Vocabulary names to exclude.

`connection` An open connection to a database server, as created by `DatabaseConnector::connect()`.

`vocabDatabaseSchema` The database and or schema where the OHDSI Vocabulary tables are loaded.

Returns: Returns an object of type Concepts with the seed concepts.

`SeedConceptFinder$clone()`: The objects of this class are cloneable with this method.

Usage:

```
SeedConceptFinder$clone(deep = FALSE)
```

Arguments:

`deep` Whether to make a deep clone.

`validateConcepts` *Validate a Concepts object*

Description

Throws an error if the object is not a valid `Concepts` object.

Usage

```
validateConcepts(concepts)
```

Arguments

`concepts`	An object of type 'Concepts'.

Value

Returns nothing. Is called for the side effect of not throwing an error.

See Also

asConcepts()

Index

asConcepts, 2
asConcepts(), *16*

ConceptAdjudicator, 3, *8*
ConceptRecommender, 4, *13*
condenseConceptSet, 5
createConceptSet, 5
createDrugConceptSet, 7

DefaultConceptAdjudicator, 8
DefaultSeedConceptFinder, 10

fetchCondenserConceptSetData, 11

getDomain, 12
getDomain(), *13*
getDomainSettings, 13

HecateConceptRecomender, 13

SeedConceptFinder, *10*, 14

validateConcepts, 15
validateConcepts(), *3*